New Relic APM — Day 2 Study Guide: Error Analytics, Alerting and Incident Response

Oct 28, 2026 · @Vivek Arora

Day 2 at a glance

Day 1 was about finding *where* time goes and *which hop* fails. Day 2 is about the work around a failure: seeing errors as **groups you own and close**, getting **alerts that fire for real problems only**, and running an **incident** from the first notification to the root cause. Everything is practised on the same easyTravel system instrumented with the New Relic Java agent on Day 1.

Agenda

Time	Session	Lab
09:30 – 09:45	Day 1 recap; check the lab system is healthy	—
09:45 – 11:00	**Module 4** — Error analytics and Errors Inbox	
11:00 – 11:15	Break	
11:15 – 12:15	Lab 2.1 — Triage a burst of exceptions and find the release that introduced it	Lab 2.1 (60 min)
12:15 – 13:00	**Module 5** — Alert conditions for production support	
13:00 – 14:00	Lunch	
14:00 – 15:00	Lab 2.2 — Turn a noisy static alert into a tuned baseline condition	Lab 2.2 (60 min)
15:00 – 15:45	**Module 6** — Incident intelligence and response workflow	
15:45 – 16:00	Break	
16:00 – 17:00	Lab 2.3 — Run a simulated P1 incident end to end	Lab 2.3 (60 min)
17:00 – 17:30	Runbook review, knowledge check, Q&A	—

Learning objectives

By the end of Day 2 you can:

1. Explain how APM agents capture errors — class, message, stack trace, attributes — and where each appears in New Relic.
2. Use **Errors Inbox** to group, assign, comment on, resolve and ignore error groups, and read *first seen*, *occurrences by version* and regression tags.
3. Separate **expected** errors from real ones with agent configuration, and explain why that matters for error rates and alerts.
4. Write NRQL that shows error rate by host, version and transaction, and pinpoint the release that introduced an error.
5. Choose between **static** and **anomaly (baseline)** thresholds for a signal, and set the aggregation window, delay, gap filling, signal loss and threshold duration deliberately.

6. Build NRQL alert conditions for business-critical and composite signals, and tune a noisy condition with priority, duration and threshold reviews.
7. Explain how **policies, conditions, incidents and issues** relate, how correlation merges issues, and how **workflows**, **destinations** and **muting rules** control who gets told and when.
8. Run an incident from notification to acknowledgement to root cause with a written runbook.

How to use this guide with the lab guide

This study guide explains the concepts; every command, click and expected screen is in the separate **New Relic APM — Day 2 Lab Guide**. Each module ends with a *"Do Lab 2.x"* pointer.

Screens in the lab guide are redrawn illustrations of the New Relic UI, labelled **Expected view**. The New Relic UI changes often: menu names may differ slightly in your account, but the workflow and the data are the same.

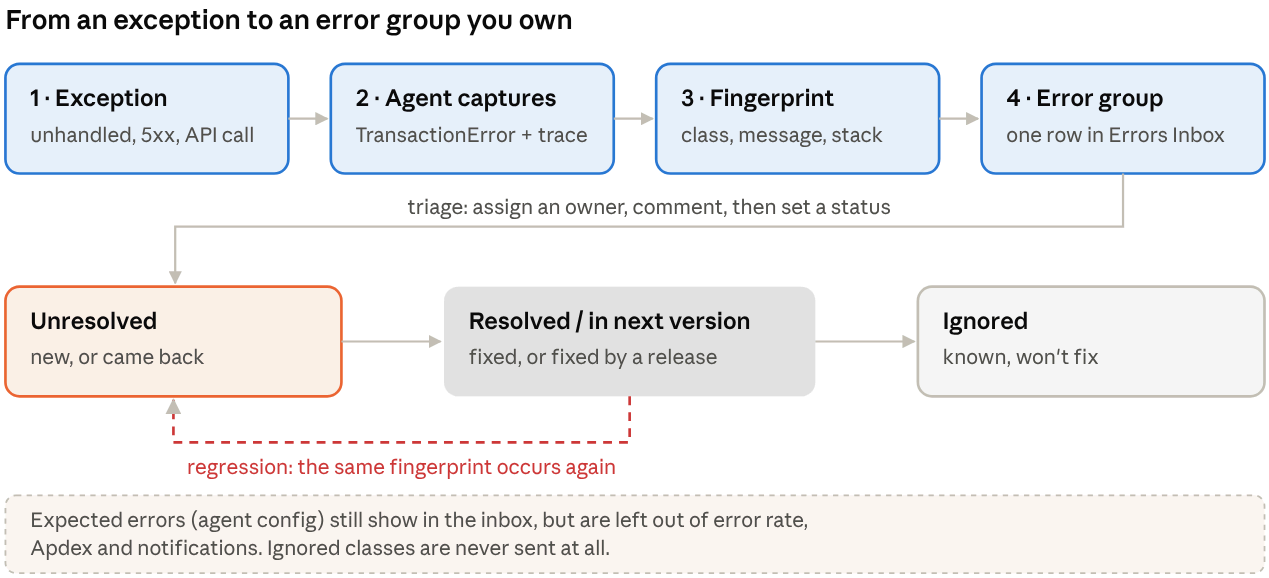
The lab system (recap)

Part	Day 2 role
`easyTravel-frontend` (Java agent)	Customer site; receives the errors users see
`easyTravel-backend` (Java agent)	Business web services; source of most exceptions
`mongodb`	Stopped on purpose in Lab 2.3 to cause the P1
`headless-customer-loadgen`	Keeps traffic flowing so errors and alerts have data
easyTravel **problem patterns**	`JourneySearchError500` (a known, noisy error), `TravellersOptionBox` (the bug shipped in release 2.2.0), `DBSpamming` (latency noise for alert tuning)
New Relic account	Same as Day 1. Day 2 also needs permission to create **alert policies, workflows, destinations and muting rules**, and a working email address for notifications

Before starting, reset the lab system: all problem patterns off, MongoDB running, distributed tracing on (Day 1 Lab Guide, Appendix).

Module 4 — Error analytics and Errors Inbox

A spike in the error rate tells you *that* something is failing. To fix it you need to know **which error**, **since when**, **where**, **for whom** and **who owns it**. Module 4 follows an error from the line of code that throws it to a group in **Errors Inbox** that someone is responsible for closing.



The error lifecycle: capture, group, triage — and the regression loop

4.1 How errors are captured

The Java agent records an error when:

- an exception **escapes** a web or background transaction (it is not caught by your code);
- a web transaction ends with an **HTTP error status** (4xx/5xx) that is not ignored — `404` is ignored by default;
- your code reports a **handled** exception with the agent API: `NewRelic.noticeError(e)` (optionally with custom attributes, or with `expected = true`).

Each error produces:

Data	Where you see it	Holds
`TransactionError` event	NRQL, Errors Inbox counts, error charts	`error.class`, `error.message`, `transactionName`, `appName`, `host`, `http.statusCode`, `error.expected`, `trace.id` / `span.id` (with distributed tracing), version tags, custom attributes
Error trace (sampled)	Errors Inbox occurrence, APM *Errors* page	The **stack trace**, request parameters and attributes of one occurrence
Span error attributes	Distributed tracing	`error.class` / `error.message` on the span where it happened
Log lines (logs in context)	Error occurrence → *Logs*	What the code logged during that request

What to look for in an occurrence, in order: the **error class and message** (what), the **top application frame of the stack trace** (where in *your* code, not the framework's), the **transaction and request attributes** (which feature, which input), the **host and version** (which instance, which release), and the linked **distributed trace** (which hop started it).

4.2 Errors Inbox: groups, triage, ownership

Errors Inbox gathers errors from APM, browser, mobile and serverless entities into one list scoped to a **workload** (a named set of entities, such as *easyTravel*) or to chosen entities.

- **Error groups.** Occurrences that share a **fingerprint** — built from the account, entity, error class, message and stack trace, with variable parts such as IDs, UUIDs and email addresses normalised away — become one group. One group = one problem to fix, however many times it happens.
- **Each group shows** occurrences over time, *users impacted*, *first seen* and *last seen*, *occurrences by version*, the most common attribute values (*profiles*), and individual occurrences with stack trace, distributed trace and logs.

- **Triage actions: assign** the group to a person (any email address), add **comments** for the team, and set a **status**: *Unresolved*, *Resolved*, *Resolve in next version* or *Ignored*. Bulk actions work on many groups at once.
- **Regressions.** When an occurrence matches the fingerprint of a resolved group, the group returns to *Unresolved* with a **regression** tag — the fix did not hold.
- **Notifications.** Errors Inbox can post new and regressed groups to Slack; expected errors are never notified.

Grouping that goes wrong. If every occurrence has a unique message ("Order 81234 failed"), you get thousands of groups. Keep variable data out of messages (put it in attributes), or set a custom grouping name with the agent's error-group callback (`error.group.name`).

Version tracking. Errors Inbox reads the release from `service.version`, `tags.service.version`, `tags.releaseTag` or `tags.commit`. For the Java agent, set environment variables and they become those tags:

```
NEW_RELIC_METADATA_SERVICE_VERSION=2.2.0      # -> tags.service.version
NEW_RELIC_METADATA_RELEASE_TAG=v2.2.0         # -> tags.releaseTag
NEW_RELIC_METADATA_COMMIT=4e1a9c0             # -> tags.commit
```

With a version attribute and change-tracking markers, *Resolve in next version* works: the group is checked again when the next release arrives, and reopened (as a regression) if it still occurs.

4.3 Expected vs unexpected errors — reducing error noise

Not every error is a defect. A `404` for an old link, a validation error for a bad card number, a business rule saying "no seats left" — these are **expected**. If they count as errors, the error rate is permanently high, real regressions hide inside it, and alerts become noise.

Choice	Agent behaviour	Use it for
Expected	Error is sent and visible (marked `error.expected = true`), but **not counted** in error rate, Apdex or error alerts, and not notified	Business errors you want to see but not be paged for
Ignored	Error is **not sent at all**	Noise with no value (bots probing URLs, client aborts)

Choice	Agent behaviour	Use it for
Fix it	—	Everything else

Java agent configuration (`newrelic.yml`, `common` section). These settings are **dynamic**: the running agent picks up a changed file without a restart.

```
error_collector:
  enabled: true
  ignore_status_codes: 404,499          # never report these
  expected_status_codes: 409             # report as expected
  expected_classes:
    - "com.example.booking.NoSeatsAvailableException"
  expected_messages:
    com.example.payment.CardValidationException:
      - "card number failed Luhn check"   # 'contains' match
  ignore_classes:
    - "org.apache.catalina.connector.ClientAbortException"
```

In code, mark a handled error as expected with `NewRelic.noticeError(e, true)`. Server-side configuration (APM *Settings → Error collection*) can also set expected and ignored classes and, when enabled, overrides the file.

> **Never mark a generic class expected.** Wrapper classes such as `org.apache.axis2.AxisFault`, `RuntimeException` or a bare `500` are shared by many unrelated failures — including your next outage. For those, use `expected_messages` (class **plus** a distinctive message phrase), fix the code to throw a specific exception, or set the group to *Ignored* in Errors Inbox.

In Errors Inbox, the default filter `error.expected != true` hides groups whose occurrences are all expected; groups with a mix are tagged *contains expected*.

4.4 Error rate trends with NRQL facets

```
-- Error rate by application over time
SELECT percentage(count(*), WHERE error IS true) AS 'error %' FROM
Transaction
WHERE appName LIKE 'easyTravel-%' FACET appName TIMESERIES SINCE 6 hours ago

-- Which hosts are failing (one bad instance vs all of them)
```

```
SELECT count(*) FROM TransactionError WHERE appName = 'easyTravel-backend'
FACET host TIMESERIES SINCE 3 hours ago

-- Which release: errors by version
SELECT count(*) FROM TransactionError WHERE appName LIKE 'easyTravel-%'
FACET tags.service.version, error.class SINCE 1 day ago

-- Which feature: errors by transaction
SELECT count(*) FROM TransactionError WHERE appName LIKE 'easyTravel-%'
FACET transactionName, error.class SINCE 1 hour ago LIMIT 20

-- When did each error class first appear? (compare with deployment markers)
SELECT earliest(timestamp) AS 'first seen', latest(timestamp) AS 'last seen',
count(*)
FROM TransactionError WHERE appName LIKE 'easyTravel-%' FACET error.class
SINCE 1 day ago

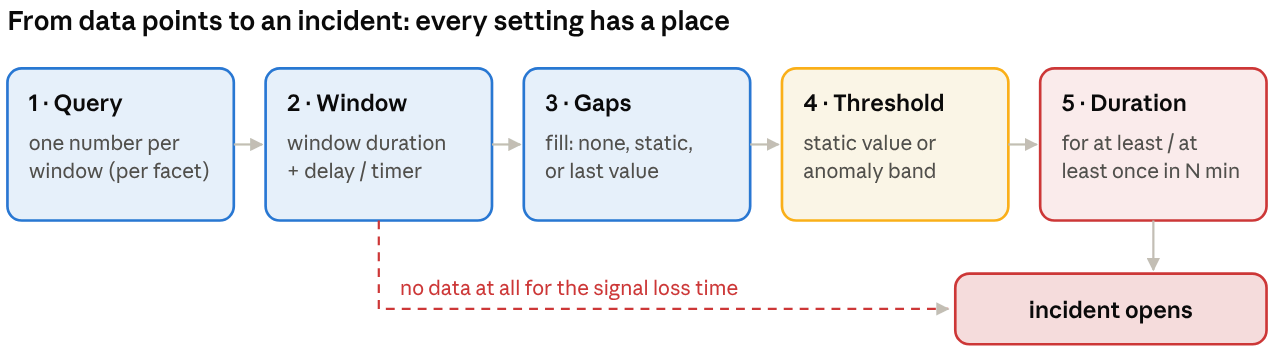
-- Unexpected errors only
SELECT count(*) FROM TransactionError WHERE error.expected IS NOT true
FACET appName, error.class SINCE 1 hour ago
```

Finding the introducing release comes down to three facts that must agree: the group's **first seen** time is just after a **deployment marker**; **occurrences by version** show the error only in the new version; and the error rate faceted by `tags.service.version` is clean for the old version. If two of the three disagree (for example, the error also occurs in the old version on one host), look for a configuration or infrastructure change instead.

> **Do Lab 2.1 — Triage a burst of exceptions** in the Day 2 Lab Guide (60 min): tag easyTravel with version 2.0.9, create an *easyTravel* workload and open Errors Inbox, ship release 2.2.0 with a bug, triage the new error group, prove which release introduced it with NRQL, mark a known error as expected, and close the group with release 2.2.1.

Module 5 — Alert conditions for production support

An alert is a promise: *when this fires, a human must act.* Every alert that fires without needing action teaches people to ignore the next one. Module 5 is about conditions that keep that promise — the right threshold type, the right signal settings, and a regular tuning habit.



The evaluation pipeline of an NRQL alert condition

5.1 Static vs anomaly (baseline) thresholds

	Static	**Anomaly** (baseline)
Fires when	The value crosses a fixed number	The value leaves a band predicted from the signal's own history
Learns	Nothing	1–4 weeks of history, weighted to recent data; seasonality *New Relic calculation*, *Hourly*, *Daily*, *Weekly* or *None*
Sensitivity	The number itself	Standard deviations from the baseline (start at 2–3; 1–2 for very stable critical signals, 3–4 for noisy ones)
Direction	Above / below / equals	*Upper only*, *Lower only*, or *Upper and lower*
Best for	Hard limits: SLA/SLO targets, error rate > 5 %, disk > 90 %, queue > capacity, *service is down*	Signals whose "normal" moves: latency through the day, throughput, booking volume, seasonal traffic
Weak at	Signals with daily or weekly cycles (fires at peak, misses at night)	Brand-new signals, rare events, error rates where any increase matters; faceted anomaly conditions learn each facet from scratch

A practical rule: **use static where the business sets the number, anomaly where the system sets it.** Many teams pair them: a static critical at the SLA limit and an anomaly warning that catches unusual drift early.

5.2 Signal settings that decide what the condition sees

Setting	What it does	How to choose
Window duration	Size of each aggregation window (30 s – 120 min; default 1 min)	Longer windows smooth noise but react later. 1–5 min for APM
Sliding window aggregation	Overlapping windows (e.g. 5-minute windows every 1 minute)	Smooths spiky signals without slowing detection as much as a long window
Streaming method	*Event flow* (default, windows close when later data arrives), *Event timer* (windows close after a timer — for sparse or batched data), *Cadence* (legacy, wall clock)	Event flow for APM, browser, logs; event timer for cloud integrations and infrequent data
Delay / timer	How long to wait for late data before evaluating a window	Default 2 min for event flow; more for cloud metrics that arrive minutes late
Fill data gaps	Value for windows with no data: *None* (gap resets the duration timer), a **static value** (often 0) or **last known value**	`0` for counts that are legitimately empty (errors); *last known value* for slow-changing gauges
Signal loss	Open (or close) an incident when the signal stops entirely for a set time (30 s – 48 h)	Heartbeats: "no transactions from the backend for 5 minutes". Must have seen the signal first
Evaluation delay	Skip evaluation for a while after a new entity appears (up to 120 min)	Autoscaling groups, rolling deploys — avoids alerts from warm-up
Threshold duration	*For at least* N minutes (every window breaches) or *At least once in* N minutes	*For at least* for sustained problems (fewer false alarms); *at least once in* only for events that must never happen

Setting	What it does	How to choose
Priority	*Critical* and optional *Warning* term per condition	Critical = page someone. Warning = look at it today

Two classic traps. (1) A `count(*)` query returns **NULL**, not 0, for windows with no matching data — so "failures above 0" never fires and gaps reset timers. Use `filter(count(*), WHERE …)` with a broader `WHERE`, or fill gaps with 0. (2) Very recent data may be incomplete; that is what the delay is for — do not set it to zero to "alert faster".

5.3 NRQL conditions for business-critical and composite signals

Infrastructure and latency alerts tell you the system hurts; **business signals** tell you customers do. In easyTravel, the critical journey is booking.

```
-- Booking failure rate (static: page above 2 %)
SELECT percentage(count(*), WHERE error IS true) FROM Transaction
WHERE appName = 'easyTravel-frontend' AND request.uri LIKE '%orange-booking%'

-- Booking volume drop (anomaly, lower only: bookings fell unusually low)
SELECT count(*) FROM Transaction
WHERE appName = 'easyTravel-frontend' AND request.uri LIKE '%orange-booking-
finish%'

-- Zero-safe failure count for a synthetic check of the booking page
SELECT filter(count(*), WHERE result = 'FAILED') FROM SyntheticCheck WHERE
monitorName = 'easyTravel booking'
```

Composite signals combine several measurements into one number, so one condition reflects the user's experience instead of three conditions firing separately:

```
-- Share of backend requests that are slow OR failed (an "unhappy requests"
ratio)
SELECT filter(count(*), WHERE duration > 1 OR error IS true) / count(*) * 100
FROM Transaction WHERE appName = 'easyTravel-backend'

-- Error budget burn: error rate divided by the SLO allowance (0.5 %)
SELECT percentage(count(*), WHERE error IS true) / 0.5 AS 'burn rate'
FROM Transaction WHERE appName = 'easyTravel-frontend'

-- Checkout health across two steps: failed payments plus failed
confirmations, as one rate
SELECT (filter(count(*), WHERE request.uri LIKE '%orange-booking-payment%'
AND error IS true)
      + filter(count(*), WHERE request.uri LIKE '%orange-booking-finish%' AND
error IS true))
      / filter(count(*), WHERE request.uri LIKE '%orange-booking%') * 100 AS
'checkout failure %'
FROM Transaction WHERE appName = 'easyTravel-frontend'
```

When the parts live in different data types (a `Transaction` signal and a `SystemSample` signal), keep them as separate conditions in the **same policy** and let the issue roll-up and correlation in Module 6 combine them.

Conditions as code (NerdGraph), useful for review and reuse:

```
mutation {
  alertsNrqlConditionBaselineCreate(accountId: 1234567, policyId: "<policy
id>", condition: {
    name: "Backend p95 latency (baseline)"
    enabled: true
    nrql: { query: "SELECT percentile(duration, 95) FROM Transaction WHERE
appName = 'easyTravel-backend'" }
    baselineDirection: UPPER_ONLY
    signalSeasonality: NEW_RELIC_CALCULATION
    terms: [{ priority: CRITICAL, operator: ABOVE, threshold: 3,
              thresholdDuration: 300, thresholdOccurrences: ALL }]
    signal: { aggregationWindow: 60, aggregationMethod: EVENT_FLOW,
aggregationDelay: 120 }
    expiration: { expirationDuration: 600, openViolationOnExpiration: false,
closeViolationsOnExpiration: true }
    violationTimeLimitSeconds: 86400
    runbookUrl: "https://wiki.example.com/runbooks/easytravel-latency"
  }) { id name }
}
```

For a static condition the mutation is `alertsNrqlConditionStaticCreate`, with `operator: ABOVE | BELOW | EQUALS` and the threshold in the query's units.

5.4 Tuning noisy alerts

A condition is **noisy** when its incidents close by themselves within minutes, fire at the same time every day, or are routinely acknowledged without action. Review every condition that pages people at least once a month:

1. **Measure.** How many incidents per condition, how many closed by themselves within ten minutes, how many led to action? (Queries below.)
2. **Check the signal.** Does the query measure what the user feels? p95 latency of the booking call is better than average CPU.
3. **Duration first.** Change *at least once in 1 minute* to *for at least 5 minutes* — the single most effective fix for flapping.
4. **Threshold type.** If incidents line up with daily peaks, move to an **anomaly** threshold with *Upper only*.
5. **Priority.** Demote "interesting but not urgent" to *Warning* and route warnings to a channel, not a pager.

6. **Window and gaps.** Lengthen the window or use a sliding window for spiky signals; fill gaps where empty windows reset timers.
7. **Context.** Add a runbook URL and a clear title template, so the person paged knows what to do.
8. **Retire.** A condition nobody acts on should be deleted, not muted forever.

```
-- Incidents opened per condition, last 7 days
SELECT count(*) FROM NrAiIncident WHERE event = 'open'
FACET policyName, conditionName SINCE 7 days ago LIMIT 50

-- When do they fire? (hour of day reveals cyclic noise)
SELECT count(*) FROM NrAiIncident WHERE event = 'open' AND conditionName =
'Backend p95 latency (static)'
FACET hourOf(timestamp) SINCE 7 days ago WITH TIMEZONE 'Asia/Kolkata'

-- Open vs close events over time for one condition (short gaps = flapping)
SELECT count(*) FROM NrAiIncident WHERE conditionName LIKE 'Backend p95%'
FACET event TIMESERIES 10 minutes SINCE 6 hours ago
```

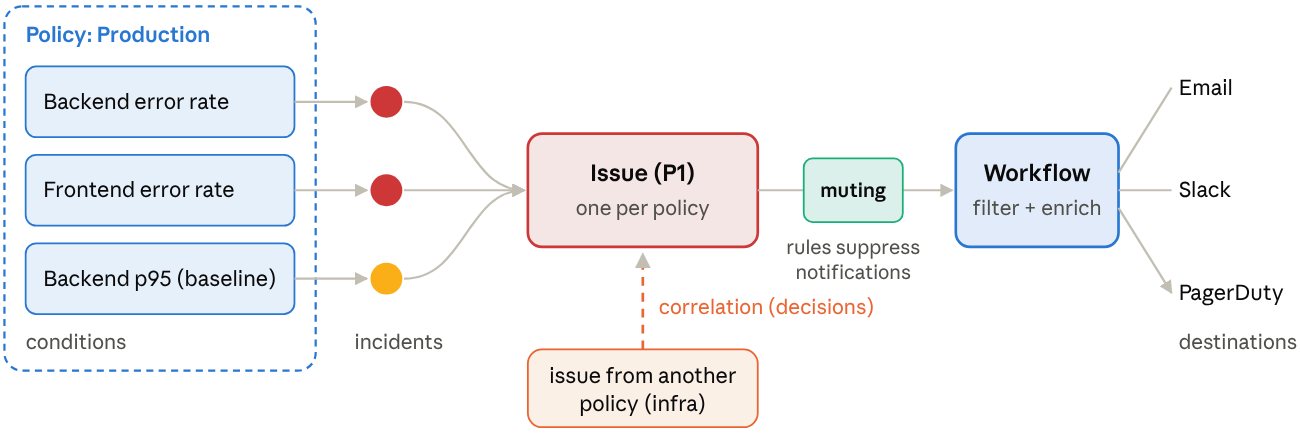
Close events also record why the incident closed (`closeCause`) and, in most accounts, how long it was open — run `SELECT keyset() FROM NrAiIncident` to see the duration attribute available to you.

> **Do Lab 2.2 — Convert a noisy static alert into a tuned baseline condition** in the Day 2 Lab Guide (60 min): create a static p95 latency condition, generate short latency blips and a real slowdown, count the noise with `NrAiIncident`, then build a baseline condition with the right window, direction, sensitivity and duration and compare the two.

Module 6 — Incident intelligence and response workflow

During an outage, one root cause can trip ten conditions across three teams. Without a model for how alerts roll up, people receive ten pages, open ten investigations and talk past each other. Module 6 covers the objects New Relic uses to turn many alerts into **one issue**, the rules that decide **who is told**, and a **runbook** for what they do next.

How alerts roll up: condition → incident → issue → workflow → people



The policy's issue creation preference decides how incidents group; correlation can merge issues across policies; workflows act on issues, never on single incidents.

The alerting hierarchy used in Lab 2.3

6.1 Policies, incidents, issues — how they roll up

Object	What it is	Example
Condition	One signal and its thresholds	*Backend error rate > 5 % for 3 min*
Policy	A group of conditions owned by one team, with an **issue creation preference**	*easyTravel Production*
Incident (alert event)	One threshold breach of one condition for one signal (one facet value)	*Backend error rate = 38 % on easyTravel-backend*
Issue	A group of incidents about the same underlying problem — the unit people acknowledge, investigate and close, and the unit that triggers notifications	*P1: easyTravel bookings failing*
Workflow	Rules that send issues matching a filter to destinations, with optional NRQL enrichments	*Critical issues in easyTravel policies → email + Slack*

Object	What it is	Example
Destination	Where notifications go: email, Slack, PagerDuty, webhook, Jira, ServiceNow, mobile app, EventBridge	*on-call@example.com*

Issue creation preference (set per policy):

Preference	Behaviour	Good for
One issue per policy (default)	All incidents of the policy go into the one open issue	A policy per service or per team — one outage, one page
One issue per condition	Each condition has its own issue	Policies that mix unrelated signals
One issue per condition and signal	Each condition *and* facet value gets its own issue	Faceted conditions where each host or customer is handled separately

An issue's **state** moves through *Activated* → *Acknowledged* (someone owns it) → *Investigating* (optional) → *Closed*; it closes when all its incidents close, or when someone closes it. Its **priority** (*Critical*, *High*, *Medium*, *Low*) follows its incidents and can be raised by hand. In NRQL, incidents are the `NrAiIncident` event type (`event = 'open'` / `'close'`), and issues are `NrAiIssue`.

6.2 Correlation and decisions — one outage, one issue

Correlation merges issues that describe the same problem, even when they come from different policies — for example the *easyTravel Production* issue and an *Infrastructure* issue about the database host. It looks at **time**, **alert context** (same condition, same entity, similar titles) and **relationships** between entities (service A calls service B).

- Turn it on per policy with **Correlate and suppress noise**, or connect policies as a source under *Alerts* → *Sources*.
- **Global decisions** are built in and on by default (for example *Same New Relic Condition* and *Topologically Dependent*).

- **Custom decisions** (*Alerts → Decisions → Create new decision*) correlate on rules you write: up to eight attribute filters, a time window (default 20 minutes, 1–120), optional similarity matching. A simulation over the last seven days shows how many issues it would have merged before you save it.
- Each merged issue says **which decision** merged it, so responders can trust — or question — the grouping.

Good custom decisions follow ownership and dependencies: *merge issues with the same* `tags.team` *within 15 minutes*, *merge any issue on a service with an issue on its database*.

6.3 Workflows, destinations and muting rules

Workflows decide who is told about which issues, and when.

- **Filter** on issue attributes: policy name, priority, entity tags (`tags.team = 'travel'`), condition names.
- **Notify when:** *Activated*, *Acknowledged*, *Investigating*, *Closed*, *Priority changed*, *Other updates*.
- **Enrich** the notification with up to two NRQL queries — for example the top error classes in the last 10 minutes — so the first message already contains evidence.
- **Test** sends a sample notification from a recent matching issue.

Muting rules stop *notifications* for known events — maintenance windows, planned failovers, load tests — while alerting keeps detecting and recording incidents (marked *Muted*). Create them under *Alerts → Muting rules*:

- **Match** on attributes such as `policyName`, `conditionName`, `entity.guid`, `targetName` or `tags.<name>` (case-sensitive), combined with AND/OR (up to 20 sub-conditions).
- **Schedule** once (start/end) or recurring daily, weekly or monthly, in a chosen time zone.
- **When the window ends:** *Notify* (recommended — the muted incident closes and, if the problem is still there, a new unmuted incident notifies) or *Suppress notification*.
- An issue that contains **only** muted incidents sends no notifications.

Prefer narrow rules (one entity or one policy, a fixed end time) to broad ones; a forgotten rule that matches everything silences real outages.

6.4 Building an incident triage runbook around New Relic signals

A runbook turns a notification into the same first fifteen minutes every time, whoever is on call:

1. **Acknowledge** the issue within the agreed time (P1: 5 minutes) — this tells everyone it is owned.
2. **Declare** severity from impact, not from the alert: which user journeys fail, for how many users (Errors Inbox *users impacted*, error rate by journey, browser sessions).
3. **Open the issue page**: correlated incidents, impacted entities, and the decision that merged them.
4. **Locate** with Day 1 tools: service map (upstream/downstream), distributed traces filtered on errors, the deepest error span, logs in context.
5. **Correlate with change**: deployment markers and change tracking, Errors Inbox *first seen* and *occurrences by version*.
6. **Mitigate** first (roll back, fail over, switch the feature off), fix second.
7. **Communicate** on a fixed cadence (P1: every 15 minutes) in one channel; record actions as you go.
8. **Close** the issue when the signals recover, and write the timeline from New Relic data (`NrAiIncident`, deployment markers, the queries you ran).

Each condition's **runbook URL** should point to the page for that signal, so step 1 starts with a link instead of a search.

> **Do Lab 2.3 — Run a simulated P1 incident** in the Day 2 Lab Guide (60 min): build the *easyTravel Production* policy and conditions, an email destination and a P1 workflow with an NRQL enrichment, test a maintenance-window muting rule, then stop MongoDB and take the issue from notification to acknowledgement to root cause and closure.

P1 incident runbook and Day 2 cheat sheet

P1 runbook (easyTravel template)

Minute	Step	In New Relic	Done when…
0–5	**Acknowledge**	Notification link → issue → *Acknowledge*	The issue shows an owner
5–10	**Size the impact**	Issue page (impacted entities, correlated incidents); error rate and throughput of the booking journey; Errors Inbox *users impacted*	Severity confirmed or downgraded; status message sent
10–20	**Locate**	Service map; distributed traces filtered on errors → deepest error span; logs in context	You can name the failing component
10–20	**Check for change**	Deployment markers, change tracking, Errors Inbox *first seen* / *occurrences by version*	A change is confirmed or ruled out
20–30	**Mitigate**	— (roll back, restart, fail over, feature off)	Signals recovering; incidents closing
every 15	**Communicate**	Post the issue link and key charts	Stakeholders informed
after	**Close and review**	Issue closes when its incidents close; timeline from `NrAiIncident`, markers and your queries	Review held, actions filed, noisy conditions tuned

Roles for a P1: **incident lead** (decides and communicates), **investigator(s)** (use New Relic to locate the cause), **scribe** (keeps the timeline). On a small team one person may hold two roles — never all three.

NRQL cheat sheet — errors (Module 4)

```
SELECT count(*) FROM TransactionError WHERE appName LIKE 'easyTravel-%' FACET error.class SINCE 1 hour ago
SELECT percentage(count(*), WHERE error IS true) FROM Transaction FACET appName TIMESERIES SINCE 6 hours ago
SELECT count(*) FROM TransactionError FACET tags.service.version, error.class SINCE 1 day ago
SELECT count(*) FROM TransactionError FACET host TIMESERIES SINCE 3 hours ago
SELECT earliest(timestamp), latest(timestamp), count(*) FROM TransactionError FACET error.class SINCE 1 day ago
SELECT count(*) FROM TransactionError WHERE error.expected IS NOT true FACET appName SINCE 1 hour ago
SELECT timestamp, error.class, error.message, transactionName, host, trace.id FROM TransactionError LIMIT 100
```

NRQL cheat sheet — alerts and incidents (Modules 5 and 6)

```
-- Incidents opened per condition
SELECT count(*) FROM NrAiIncident WHERE event = 'open' FACET policyName, conditionName SINCE 7 days ago

-- Incident timeline for a window (post-incident review)
SELECT timestamp, event, priority, conditionName, title, closeCause FROM NrAiIncident
WHERE policyName = 'easyTravel Production' SINCE 2 hours ago ORDER BY timestamp ASC LIMIT MAX

-- Flapping: open/close events in 10-minute buckets
SELECT count(*) FROM NrAiIncident WHERE conditionName = '<condition>' FACET event TIMESERIES 10 minutes SINCE 1 day ago

-- Muted incidents
SELECT count(*) FROM NrAiIncident WHERE event = 'open' AND muted IS true FACET conditionName SINCE 1 day ago

-- Issues (attributes vary — run keyset() first)
SELECT keyset() FROM NrAiIssue SINCE 7 days ago

-- What an anomaly condition saw (signal values and band)
SELECT keyset() FROM NrAiSignal WHERE conditionId = <condition id> SINCE 1 hour ago
```

Alert condition quick reference

Signal	Threshold type	Starting settings
Error rate (%)	Static	> 5 % critical, > 2 % warning, *for at least* 3–5 min, window 1 min
p95 latency	Anomaly, *Upper only*	3 std dev critical, 2 warning, *for at least* 5 min
Throughput	Anomaly, *Lower only*	3 std dev, *for at least* 10 min
Service heartbeat	Static + **signal loss**	`count(*)` of transactions, signal lost after 5 min opens an incident
Synthetic failures	Static	`filter(count(*), WHERE result = 'FAILED')` > 0, window 5 min, gaps filled with 0
Disk used (%)	Static	> 90 % critical, latest() per host and mount, *for at least* 10 min

Knowledge check, answers and sources

Questions

1. Name three ways an error reaches New Relic from a Java service.
2. Errors Inbox shows 4,000 groups for one service, each with one or two occurrences. What is the likely cause and the fix?
3. What is the difference between an **expected** and an **ignored** error? Which one still appears in Errors Inbox?
4. Which three facts together prove that release 2.2.0 introduced an error?
5. What does *Resolve in next version* need in order to work, and what happens if the error appears in the next release?
6. Latency peaks every day at 10:00 and a static p95 condition fires every morning. What would you change?
7. A condition `SELECT count(*) FROM TransactionError WHERE appName = 'checkout'` with threshold "above 0 *for at least* 5 minutes" never fires, although errors arrive in bursts every couple of minutes. Why, and what would you change?

8. When would you use *Event timer* instead of *Event flow*?
9. What does **signal loss** detect that a normal threshold cannot? Give a condition where it is the right tool.
10. A policy has three conditions, all breached by the same outage. How many issues are opened with *One issue per policy*? With *One issue per condition and signal*, for a condition faceted by host on 4 hosts?
11. What does a muting rule suppress, and what does it not?
12. Put these in order for a P1: mitigate, acknowledge, locate, size the impact, check for change, close and review.

Answers

1. An unhandled exception that leaves a transaction; an HTTP error status (4xx/5xx, except ignored codes such as 404); a handled error reported with `NewRelic.noticeError()`.
2. Error messages contain variable data (IDs, amounts, timestamps), so every occurrence gets its own fingerprint. Move the variable parts into attributes, or set a custom grouping name (`error.group.name`) through the agent's error-group callback.
3. **Expected** errors are sent and visible (`error.expected = true`) but excluded from error rate, Apdex and notifications; **ignored** errors are dropped by the agent and never sent. Expected errors still appear in Errors Inbox (hidden by the default filter).
4. The group's *first seen* is just after the 2.2.0 deployment marker; *occurrences by version* (or `FACET tags.service.version`) show it only in 2.2.0; and the previous version is clean for the same traffic.
5. A version attribute on the errors (for Java: `NEW_RELIC_METADATA_SERVICE_VERSION` → `tags.service.version`, or release tag / commit) and change tracking of releases. If it occurs in the next or a later version, the group is reopened as *Unresolved* with a **regression** tag.
6. Switch to an **anomaly** threshold (*Upper only*, seasonality *New Relic calculation* or *Daily*), and use *for at least* 5 minutes rather than *at least once in*.
7. Windows between the bursts have no `TransactionError` rows, so the query returns no value; empty windows reset the *for at least* timer, so five breaching windows in a row never happen. Query a data type that always has rows — `SELECT filter(count(*), WHERE error IS true) FROM Transaction WHERE appName = 'checkout'` returns 0 between bursts — or fill gaps with 0, and consider an error *rate* instead of a count.

8. For sparse or batched data (cloud integrations, infrequent jobs) where windows should close on a timer instead of waiting for later data points.
9. That a signal **stopped completely** (no data at all) — thresholds evaluate values, and there are none. Example: `SELECT count(*) FROM Transaction WHERE appName = 'easyTravel-backend'` with "signal is lost after 5 minutes → open an incident" for a service heartbeat.
10. One issue (all incidents join the policy's open issue). With *one issue per condition and signal*: one issue for each condition/signal pair — the faceted condition alone opens four (one per host), plus one per other breached signal.
11. It suppresses **notifications** for matching incidents and issues; it does not stop detection — incidents are still opened, recorded and marked *Muted*.
12. Acknowledge → size the impact → locate → check for change (in parallel with locating) → mitigate → close and review.

Sources

- New Relic docs — Errors inbox and Version tracking
- New Relic docs — Java agent: error collector settings and Manage errors: collect, ignore or mark expected
- New Relic docs — Create NRQL alert conditions and Anomaly detection
- New Relic docs — Specify when alerts create incidents
- New Relic docs — Correlation logic with decisions
- New Relic docs — Workflows and Muting rules
- New Relic docs — Alert event attributes (NrAiIncident)
- Dynatrace — easyTravel problem patterns

New Relic's UI changes often; screen and menu names in this guide follow the documentation above at the time of writing and may differ slightly in your account.